SAGE FOUNDATION

OPEN-SOURCE SDK + FIRMWARE ALPHA

Meshticulum
Testers' Guide

Try the desktop example, build firmware if you have a matching board, and tell us what worked or got in your way. Meshticulum is an experimental open-source SDK and firmware alpha. The commands below use a macOS or Linux shell.

1. Try it on your computer

You need Git, Python 3.11 or later, and internet access to install dependencies. This example works without a radio, Lantern or SAGE. This copy uses the tested source commit below.

TERMINAL

```
git clone https://github.com/l33tdawg/meshticulum.git
cd meshticulum
git fetch origin eba5991a2113fb9a8e33f8f4348670efe3ff6baf
git switch --detach FETCH_HEAD
git rev-parse HEAD
python3 -m venv .venv
. .venv/bin/activate
python -m pip install .
python -I - <<'PY'
from importlib.metadata import version
import meshticulum
print(version("meshticulum"))
print(meshticulum.__file__)
PY
meshticulum-seed --help
python examples/offline_resource_demo.py
```

Keep the commit printed by `git rev-parse HEAD`. The import check should show `1.0.0a1` and a path in your virtual environment's `site-packages`. CLI help should list `serve`, `serve-cache` and `fetch`; it starts no service.

The example should finish with `Verified public chunk: 287 bytes`. It creates a temporary public pack, signs the manifest with a disposable test key, then checks the signature and returned content locally. It opens no network or device connection. The SDK guide explains the code. To test the released wheel instead of the current checkout, use the alpha installation instructions and verify the download against the release's SHA256SUMS.

2. Build firmware for a matching board

This recipe requires **Apple Silicon macOS**; it refuses other host platforms. The recorded build used Python 3.12.8 and PlatformIO Core 6.1.19. The recipe pins PlatformIO and downloads 14 checksum-checked input archives. Linux or Intel hosts need an adapted, reviewed recipe. The documented post-flash inspection and radio example run on Linux, separately from this build host.

Your hardware	Recipe target	Recorded scope
Heltec LoRa 32 V4, ESP32-S3	`mtbench_heltec_v4`	Build, application installation and radio test evidence
Seeed XIAO ESP32-S3 with WIO-SX1262	`mtbench_xiao_s3_wio`	Build evidence; physical acceptance remains open

A matching radio chip alone does not establish board compatibility. From the repository root, with a fresh `build/firmware-work` path:

TERMINAL

```
python3.12 -m venv build/firmware-cli
build/firmware-cli/bin/python -m pip install \
  -r firmware/meshticulum-bench/requirements-firmware.txt
python3 firmware/meshticulum-bench/prepare_firmware.py fetch \
  --archive-dir build/firmware-archives
python3 firmware/meshticulum-bench/prepare_firmware.py prepare \
  --archive-dir build/firmware-archives \
  --output-dir build/firmware-work
python3 firmware/meshticulum-bench/prepare_firmware.py build \
  --output-dir build/firmware-work \
  --pio-python build/firmware-cli/bin/python \
  --target mtbench_heltec_v4
```

For XIAO, use `--target mtbench_xiao_s3_wio`. Omitting `--target` builds both. Keep failed logs; use a separately named fresh output directory for a repeat preparation/build. Do not delete evidence or bypass a checksum failure.

A successful build records return codes and output hashes in `build/firmware-work/evidence/BUILD.json`. Keep that file and `build/firmware-work/SOURCE.json`. Target outputs are in `build/firmware-work/project/.pio/build/<target>/`; retain the BIN, ELF, map, bootloader and partition files. Use `SOURCE.json.identity` as this build's expected application version. See the full recipe.

3. Flash, then check the device

There is no prebuilt firmware download. Choose the procedure for your actual board and current contents:

• **Spare development board whose contents you can replace:** follow first installation. It writes the bootloader, partition table, OTA selector and application. Use the exact target's files and confirm its physical identity and layout.

• **Board with a setup you need to preserve:** follow app-only flashing and rollback. It requires a trusted copy of the current application, checks the installed layout and saves the exact sectors needed to undo the application write. This backup does not undo a first install's layout changes. Complete the backup and durable evidence steps before writing. If those checks refuse, get a board-specific procedure.

Both procedures use esptool 4.5.1. The app-only guide also uses GNU `timeout`; macOS does not supply that command by default. Keep exclusive control of the selected serial port and follow the chosen procedure through verification. Do not mix its commands with the other installation path.

For a Heltec receiver, follow the firmware identity preparation before post-flash inspection. Give preparation the full `build/firmware-work/SOURCE.json` and its SHA-256. Use the resulting compact `identity.json` and its own hash in `inspect`, as shown in the flashing guide; hardware commands accept at most 64 KiB of metadata. Keep the full build manifest too. Inspection runs on Linux with the board's stable `/dev/serial/by-id` path. Expect the exact board/version, a fresh boot, and disarmed, non-transmitting, unfenced status. If an app-only replacement fails to boot or inspect, retain the evidence and follow the same-board rollback procedure using its checked backup; report an unresolved restore. A failed first installation needs the board-specific recovery procedure, not that app-only backup.

Ready for a radio transfer? The signed-file walkthrough uses two Linux hosts with the documented Orange Pi 5 Plus/SX1262 HAT sender and Heltec V4 receiver. It requires your application's peer keys, finite grants, regional/antenna checks and exclusive device ownership. The current bench profiles are bounded to 902-928 MHz with bandwidth margins; a country field does not add another band or confer RF permission. Read the prerequisites before starting: the receiver can transmit acknowledgments too. This firmware speaks Meshticulum's protocol; existing MeshCore and Reticulum clients are not wire-compatible.

4. Send a useful result

Open an issue with the smallest test that shows the problem. Successful test notes and suggestions are useful too. Copy this starter and fill in the parts you used:

RESULT TEMPLATE

```
Test: desktop / firmware build / flash / radio
Commit (git rev-parse HEAD) or release + package version:
Host OS, CPU architecture and Python version:
Board model, recipe target and firmware identity (if used):
Exact commands and steps:
Expected result:
Actual result, exit code and relevant log:
Source/build/output hashes (if used):
For radio: profile, input size/hash, timing boundaries,
receipt/content checks and whether both endpoints closed:
```

Remove private keys, shared secrets, credentials and personal data from logs; mask full device serial numbers and precise locations too. Keep the original evidence locally. Bug-fix PRs and documentation improvements are welcome; include how you reproduced and checked the change.

For help getting started, email Wick at SAGE Foundation with your board and host details. Project | SAGE Foundation